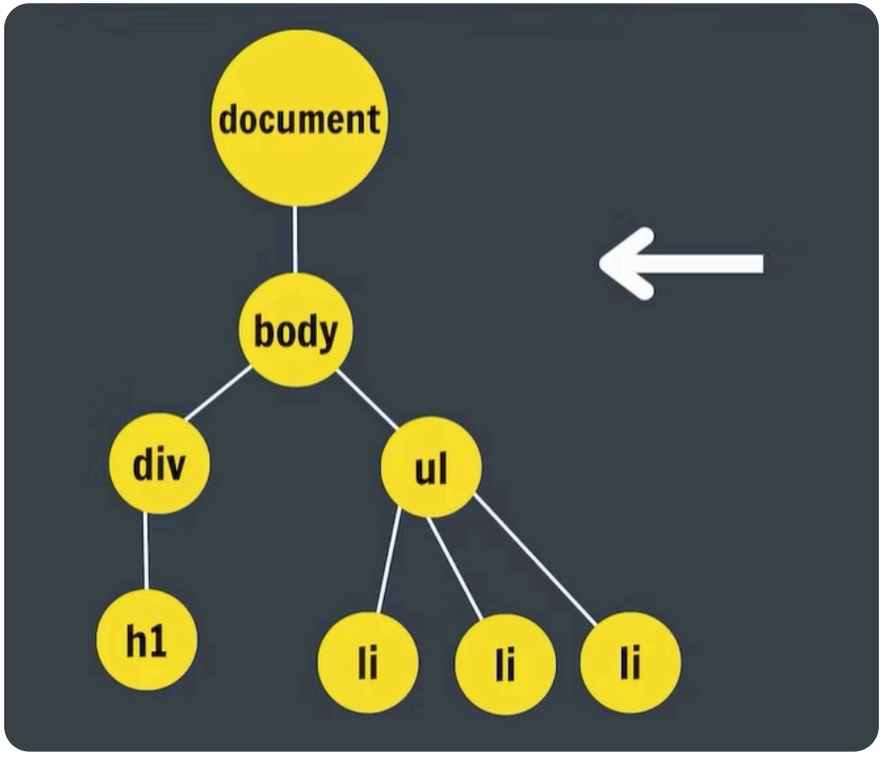
# DOM (Document Object Model)

The DOM represents a document with a logical tree.

It allows us to manipulate/change webpage content (HTML elements).




(HTML + CSS)

```
<body>
    <div>
        <h1>Todo</h1>
    </div>
    <ul>
        <li>Eat</li>
        <li>Code</li>
        <li>Sleep</li>
    </ul>
</body>
```

```
> window
<· ▶ Window {0: Window, window: Window, self: Window, docu
  ment: document, name: '', location: Location, …}
> document
<· ▶#document
> console.log(document);
  ▶#document                                          VM263:1
<· undefined
> console.dir(document);
                                                      VM361:1
  ▼#document
    ▶location: Location {ancestorOrigins: DOMStringList,
     URL: "chrome://new-tab-page/"
    ▶activeElement: body
    ▶adoptedStyleSheets: Proxy(Array) {}
     alinkColor: ""
    ▶all: HTMLAllCollection(139) [html, head, meta, titl
    ▶anchors: HTMLCollection []
    ▶applets: HTMLCollection []
     baseURI: "chrome://new-tab-page/"
     bgColor: ""
```

# DOM Manipulation: A Two-Step Process

## Step 1: Selection

The first step in DOM manipulation is to identify and select the specific HTML element(s) you want to interact with. This involves telling the browser exactly which part of the document you intend to modify.

- `getElementById()`: Selects a single element by its unique `id` attribute.
- `querySelector()`: Returns the first element that matches a specified CSS selector.
- `querySelectorAll()`: Returns a static `NodeList` of all elements that match a specified CSS selector.
- `getElementsByClassName()`: Returns an HTMLCollection of elements with the given class name.
- `getElementsByTagName()`: Returns an HTMLCollection of all elements with the given tag name.

## Step 2: Manipulation

Once an element or a collection of elements has been selected, the next step is to modify their properties, content, or structure. This is where you dynamically change the appearance or behavior of your web page.

- Changing content: Use `innerHTML` to get/set HTML content or `textContent` for just text.
- Modifying styles: Directly access and change CSS properties via the `style` property (e.g., `element.style.color = 'blue'`).
- Adding/removing classes: Use the `classList` API (`add()`, `remove()`, `toggle()`) to manage CSS classes.
- Setting attributes: Use `setAttribute()` to add or change any HTML attribute.
- Adding/removing elements: Use methods like `appendChild()`, `insertBefore()`, `removeChild()`, or `replaceChild()`.

```
> window
<· ▶ Window {window: Window, self: Window, document:
  document, name: '', location: Location, …}
> console.dir(document);
  ▶ #document                                    VM599:1
<· undefined
> console.dir(document.all);
  ▶ HTMLAllCollection(38)                        VM1969:1
<· undefined
> console.dir(document.all[8]);
  ▶ h1                                           VM2381:1
<· undefined
> console.dir(document.all[8].innerText);
  Spider Man                                     VM2411:1
<· undefined
> document.all[8].innerText = "Peter Parker";
```

# Selecting Elements

## getElementById

Returns the Element as an object or null (if not found)

```
> document.getElementById(mainImg);
<· null
> document.getElementById("mainImg");
<·   <img src="assets/spiderman_img.png" id="mainImg">
> let imgObj = document.getElementById("mainImg");
<· undefined
> imgObj
<·   <img src="assets/spiderman_img.png" id="mainImg">
```

```
> imgObj.tagName
<· 'IMG'
> imgObj.id
<· 'mainImg'
> imgObj.src = "assets/creation_1.png";
```

# Selecting Elements

## getElementByClassName

**Returns the Elements as an HTML Collection or empty collection (if not found)**

```
> document.getElementsByClassName("oldImg");
<· ▼ HTMLCollection(3) [img.oldImg, img.oldImg, img.oldImg] i
    ▶ 0: img.oldImg
    ▶ 1: img.oldImg
    ▶ 2: img.oldImg
      length: 3
    ▶ [[Prototype]]: HTMLCollection
```

```
1   let smallImages = document.getElementsByClassName("oldImg");
2
3   for (let i = 0; i < smallImages.length; i++) {
4       smallImages[i].src = "assets/spiderman_img.jpg";
5       console.log(`value of image no. ${i} is changed.`);
6   }
7
```

# Selecting Elements

## getElementByTagName

Returns the Elements as an HTML Collection or empty collection (if not found)

```
> document.getElementsByTagName("p");
<· ▶ HTMLCollection(2) [p, p#description, description: p#description]
> document.getElementsByTagName("p")[1]
<· p#description
```

# Query Selectors

**Allows us to use any CSS selector**

```
document.querySelector('p'); //Selects first p element

document.querySelector('#myId'); //Selects first element with id = myId

document.querySelector('.myClass'); //Selects first element with class = myClass
```

```
document.querySelectorAll("p"); //Selects all p elements
```

```
// console.dir(document.querySelector("p"));

// console.dir(document.querySelector("#description"));

// console.dir(document.querySelector(".oldImg"));

console.dir(document.querySelector("div a"));
```

# Using Properties & Methods

**innerText**

Shows the visible text contained in a node

**textContent**

Shows all the full text

**innerHTML**

Shows the full markup

```
> para.innerText;
<· 'Spider-Man is a superhero appearing in American
   comic books published by Marvel Comics. Created b
   y writer-editor Stan Lee and artist Steve Ditko,
   he first appeared in the anthology comic book Ama
   zing Fantasy #15 (August 1962) in the Silver Age
   of Comic Books. He has been featured in comic boo
   ks, television shows, films, video games, novels,
   and plays'
> para.innerHTML;
<· '\n      <b>Spider-Man</b> is a superhero appeari
   ng in American comic books\n      published by\n
   <a href="https://www.google.com/search?q=Marvel+C
   omics">Marvel Comics</a>.\n      Created by write
   r-editor <b>Stan Lee</b> and artist <b>Steve Ditk
   o</b>, he\n      first appeared in the anthology
   comic book\n      <a href="https://www.google.co
   m/search?q=Amazing+Fantasy">Amazing Fantasy</a>\n
   #15 (August 1962) in the <a href="#">Silver Age o
   f Comic Books</a>. He has\n      been featured in
   comic books, television shows, films, video game
   s,\n      novels, and plays\n      '
> para.textContent;
<· '\n      Spider-Man is a superhero appearing in A
   merican comic books\n      published by\n      Ma
   rvel Comics.\n      Created by writer-editor Stan
   Lee and artist Steve Ditko, he\n      first appea
   red in the anthology comic book\n      Amazing Fa
   ntasy\n      #15 (August 1962) in the Silver Age
```

```
> para.innerText = "abc";
<· 'abc'
> para.innerText = "Hi, I am Peter Parker!";
<· 'Hi, I am Peter Parker!'
> para.innerText = "Hi, I am <b>Peter Parker</b>!";
<· 'Hi, I am <b>Peter Parker</b>!'
> para.innerText = "Hi, I am <b>Peter
  Parker</b>!";
```

```
> let heading = document.querySelector('h1');
<· undefined
> heading.innerText;
<· 'Spider Man'
> heading.innerHTML = ''
✕ Uncaught SyntaxError: Invalid or          VM3393:1
  unexpected token
> heading.innerHTML = "<u>Spider Man</u>
```

or

```
> heading.innerHTML =`<u>${heading.innerText}</u>`;
<· '<u>Spider Man</u>'
```

# Manipulating Attributes

obj.getAttribute( attr )

obj.setAttribute( attr, val )

```
> let img = document.querySelector('img');
<· undefined
> img
<·   <img src="assets/spiderman_img.png" id="mainIm
     g">
> img.getAttribute('id');
<· 'mainImg'
> img.setAttribute('id', 'spidermanImg');
<· undefined
```

# Manipulating Style

**style Property**

**obj.style**

```
> let img = document.querySelector('img');
<· undefined
> console.dir(img);
  ▶ img#mainImg                                  VM4272:1
<· undefined
> img.style
<·   CSSStyleDeclaration {accentColor: '', additive
  ▶ Symbols: '', alignContent: '', alignItems: '',
    alignSelf: '', …}
> let heading = document.querySelector('h1');
<· undefined
> heading.style;
<·   CSSStyleDeclaration {accentColor: '', additive
  ▶ Symbols: '', alignContent: '', alignItems: '',
    alignSelf: '', …}
> heading.style.color = 'purple';
<· 'purple'
> heading.style.color = 'green';
<· 'green'
> heading.style.backgroundColor = 'green';
```

```
let links = document.querySelectorAll(".box a");

for(link of links) {
  link.style.color = "purple";
}

// for (let i = 0; i < links.length; i++) {
//   links[i].style.color = "green";
// }
```

obj.classList

classList.add( ) to add new classes

classList.remove( ) to remove classes

classList.contains( ) to check if class exists

classList.toggle( ) to toggle between add & remove

```
> let heading = document.querySelector('h1');
<· undefined
> heading.classList.add("green");
<· undefined
>
```

```
> heading.classList
<· ▶ DOMTokenList ['green', value: 'green']
> heading.classList.contains("underline");
<· false
> heading.classList.contains("green");
<· true
```

```
> heading.classList.toggle("green");
<· false
> heading.classList
<· ▶ DOMTokenList [value: '']
> heading.classList.toggle("underline");
<· true
> heading.classList.toggle("underline");
```

# Navigation

parentElement

children

previousElementSibling / nextElementSibling

```
> let h4 = document.querySelector('h4');
<· undefined
> h4.parentElement
<· ▶<div class="box">…</div>
> h4.children
<· ▶ HTMLCollection []
> let box = document.querySelector('.box');
<· undefined
> box.children;
<· ▶ HTMLCollection(2) [h4, ul]
> box.childElementCount;
<· 2
> let ul = document.querySelector('ul');
<· undefined
> ul.parentElement
<· ▶<div class="box">…</div>
> ul.childElementCount;
<· 3
> ul.children;
<· ▶ HTMLCollection(3) [li, li, li]
```

```
> ul.children[2];
<· ▶<li>…</li>
> ul.children[2].previousElementSibling;
<· ▶<li>…</li>
> ul.children[1].previousElementSibling;
<· ▶<li>…</li>
> ul.children[1].nextElementSibling;
<· ▶<li>…</li>
>
```

```
> let img = document.querySelector('img');
<· undefined
> img.previousElementSibling;
<·   <h1>Spider Man</h1>
> img.previousElementSibling.style;
<·   CSSStyleDeclaration {accentColor: '', additive
  ▶ Symbols: '', alignContent: '', alignItems: '',
    alignSelf: '', …}
> img.previousElementSibling.style.color = "green";
<· 'green'
>
```

# Adding Elements

document.createElement('p')

- appendChild( element )
- append( element )
- prepend( element )
- insertAdjacent( where, element)

```
> document.createElement('p');
<-   <p></p>
> let newP = document.createElement('p');
<- undefined
> console.dir(newP);
  ▶ p                                          VM8278:1
<- undefined
> newP.innerText = "Hi, I am a new p";
<- 'Hi, I am a new p'
> console.dir(newP);
  ▶ p                                          VM8376:1
<- undefined
> let body = document.querySelector('body');
<- undefined
> body.appendChild(newP);
<-   <p>Hi, I am a new p</p>
>
```

```
> newP
<-   <p>Hi, I am a new p</p>
> let box = document.querySelector('.box');
<- undefined
> box.appendChild(newP);
<-   <p>Hi, I am a new p</p>
> let btn = document.createElement('button');
<- undefined
> console.dir('btn');
  btn                                          VM8844:1
<- undefined
> console.dir(btn);
  ▶ button                                     VM8852:1
<- undefined
> btn.innerText = "click me!";
<- 'click me!'
> box.appendChild(btn);
<-   <button>click me!</button>
>
```

```
> newP
<-   <p>Hi, I am a new p</p>
> newP.append("this is new text");
<- undefined
> newP.append(btn);
<- undefined
> newP.append(" do not click this button");
<- undefined
> b
```

**position**

A string representing the position relative to the `targetElement`; must match (ca
one of the following strings:

- `'beforebegin'`: Before the `targetElement` itself.
- `'afterbegin'`: Just inside the `targetElement`, before its first child.
- `'beforeend'`: Just inside the `targetElement`, after its last child.
- `'afterend'`: After the `targetElement` itself.

```
> btn
<-   <button>NEW BUTTON!!!</button>
> p
<- ▶<p> … </p>
> p.insertAdjacentElement('beforebegin', btn);
<-   <button>NEW BUTTON!!!</button>
> p.insertAdjacentElement('afterbegin', btn);
<-   <button>NEW BUTTON!!!</button>
> p.insertAdjacentElement('beforeend', btn);
<-   <button>NEW BUTTON!!!</button>
> p.insertAdjacentElement('afterend', btn);
```

# Removing Elements

- removeChild( element )  appendChild
- remove( element )

```
> body.removeChild(btn);
<·   <button>NEW BUTTON!!!</button>
> p.insertAdjacentElement('afterend', btn);
<·   <button>NEW BUTTON!!!</button>
> btn.remove();
<· undefined
```

# Practice Qs

Add the following elements to the container using only JavaScript and the DOM methods.

i) a <p> with red text that says “Hey I’m red!”

ii) an <h3> with blue text that says “I’m a blue h3!”

iii) a  with a black border and pink background color with the following elements inside of it:

- another <h1> that says “I’m in a div”
- a <p> that says “ME TOO!”